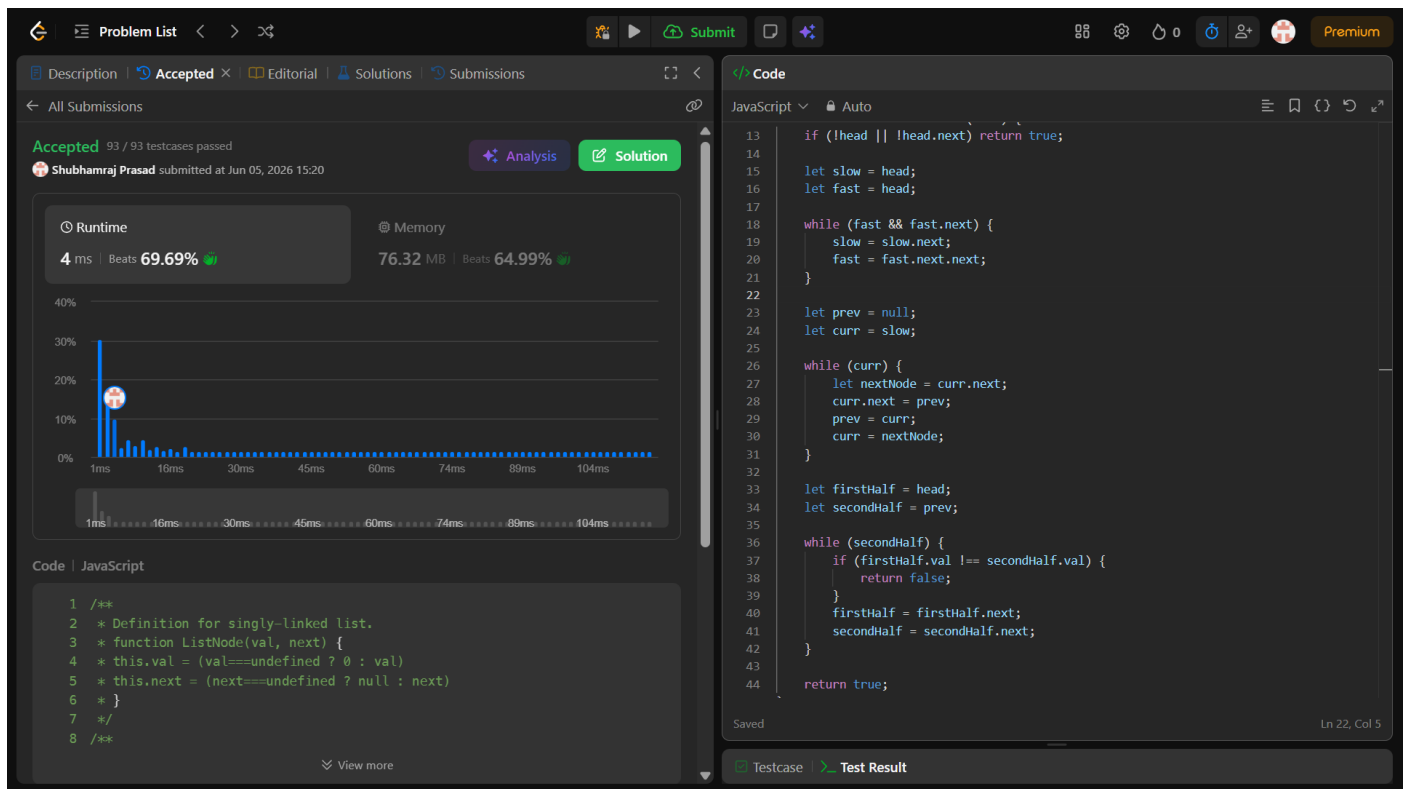


Question 1:- <https://leetcode.com/problems/palindrome-linked-list/>

Solution: - <https://leetcode.com/problems/palindrome-linked-list/submissions/2023143057/>

Screenshot:-



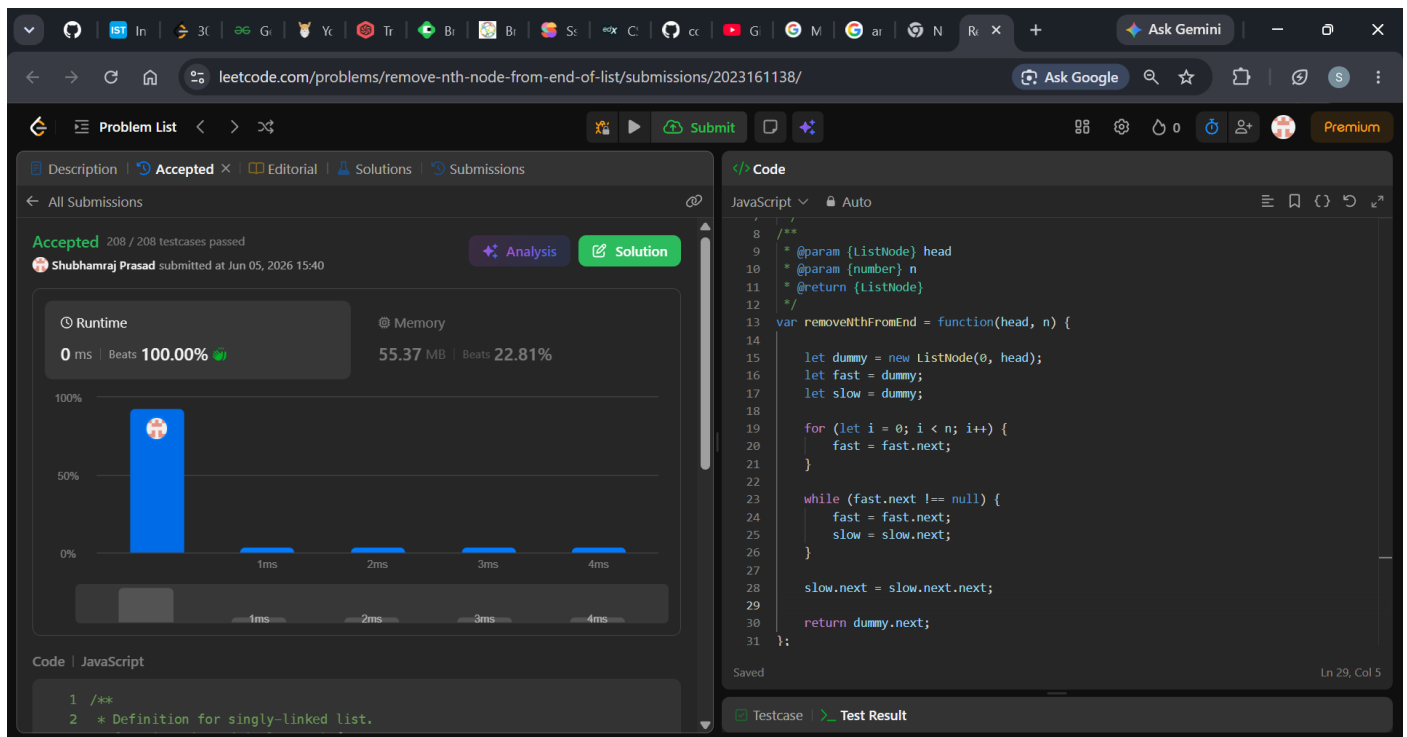
Description: -

- **Time Complexity:** $O(n)$ — Finding the middle takes $O(n/2)$ time, reversing the second half takes $O(n/2)$ time, and comparing the two halves takes at most $O(n/2)$ operations. This simplifies to a linear time complexity.
- **Space Complexity:** $O(1)$ — The mutations and reversals are performed entirely in-place by changing the node pointers, modifying no extra memory allocations.

Question 2:- <https://leetcode.com/problems/remove-nth-node-from-end-of-list/>

Solution: - <https://leetcode.com/problems/remove-nth-node-from-end-of-list/submissions/2023161138/>

Screenshot:-



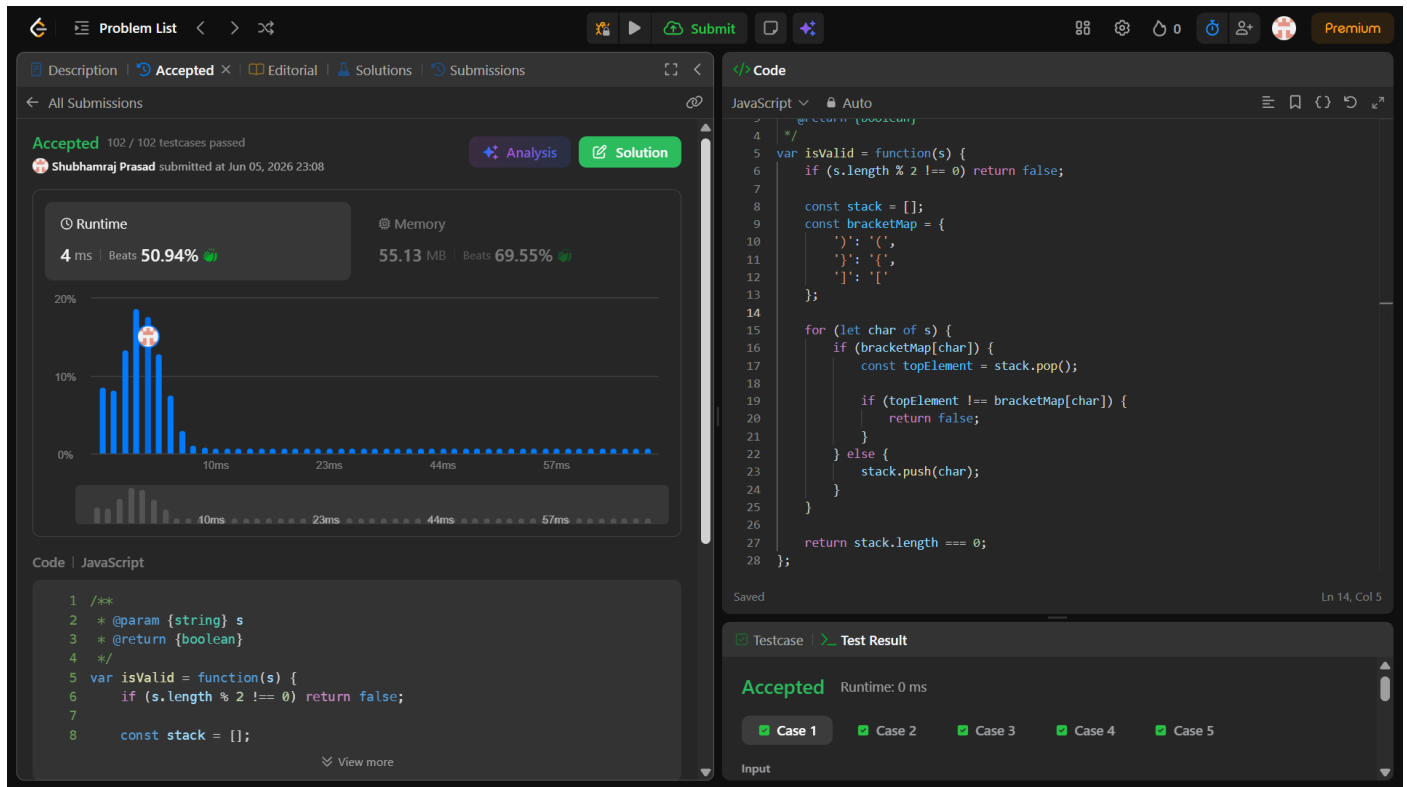
Description: -

- **Time Complexity:** $O(M)$ where M is the number of nodes in the linked list. The list is traversed exactly once.
- **Space Complexity:** $O(1)$ because we only use a few pointer variables, satisfying the optimal memory constraint.

Question 3:- <https://leetcode.com/problems/valid-parentheses/description/>

Solution: - <https://leetcode.com/problems/valid-parentheses/submissions/2023530890/>

Screenshot:-



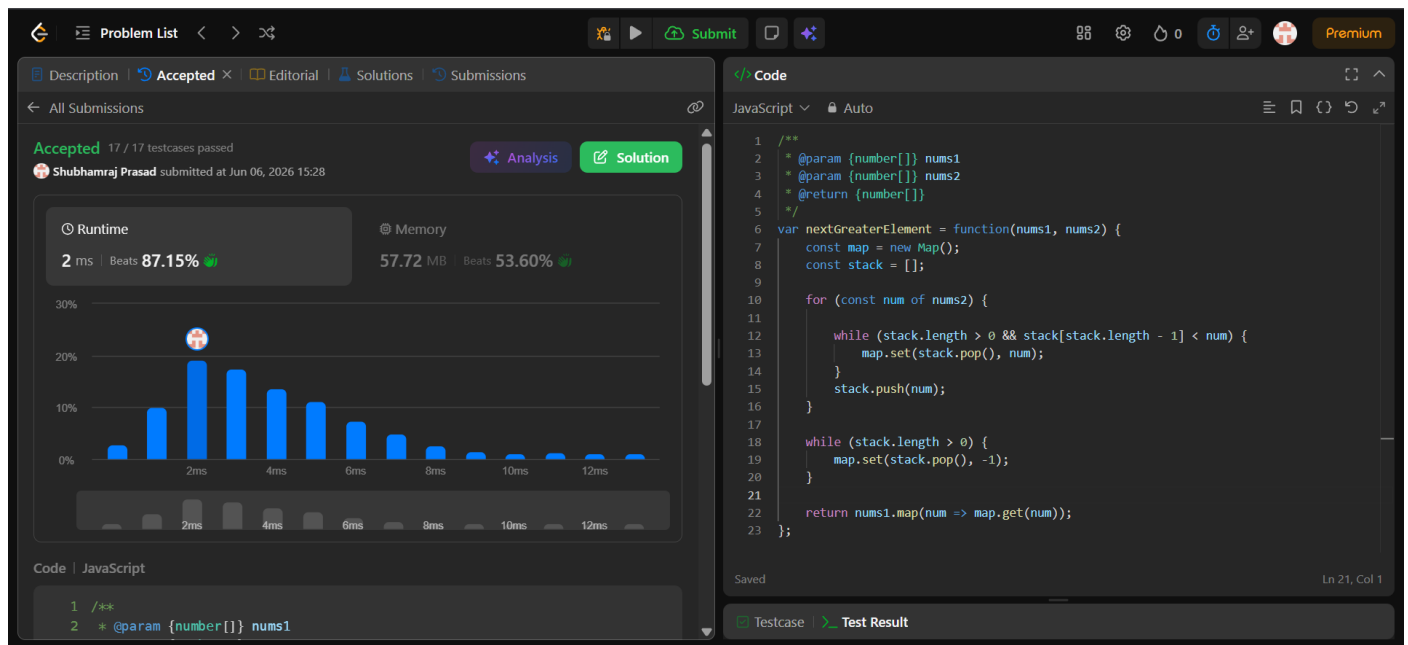
Description: -

- **Time Complexity: $O(n)$** - Linear time. The code iterates through the string of length n exactly once. Hash map lookups, pushing, and popping elements take constant $O(1)$ time per character.
- **Space Complexity: $O(n)$** - Linear space. In the worst-case scenario (e.g., an input with all opening brackets like "((((("), every character is pushed onto the stack, making its size proportional to the input length.

Question 4:- <https://leetcode.com/problems/next-greater-element-i/description/>

Solution: - <https://leetcode.com/problems/next-greater-element-i/submissions/2024098649/>

Screenshot:-



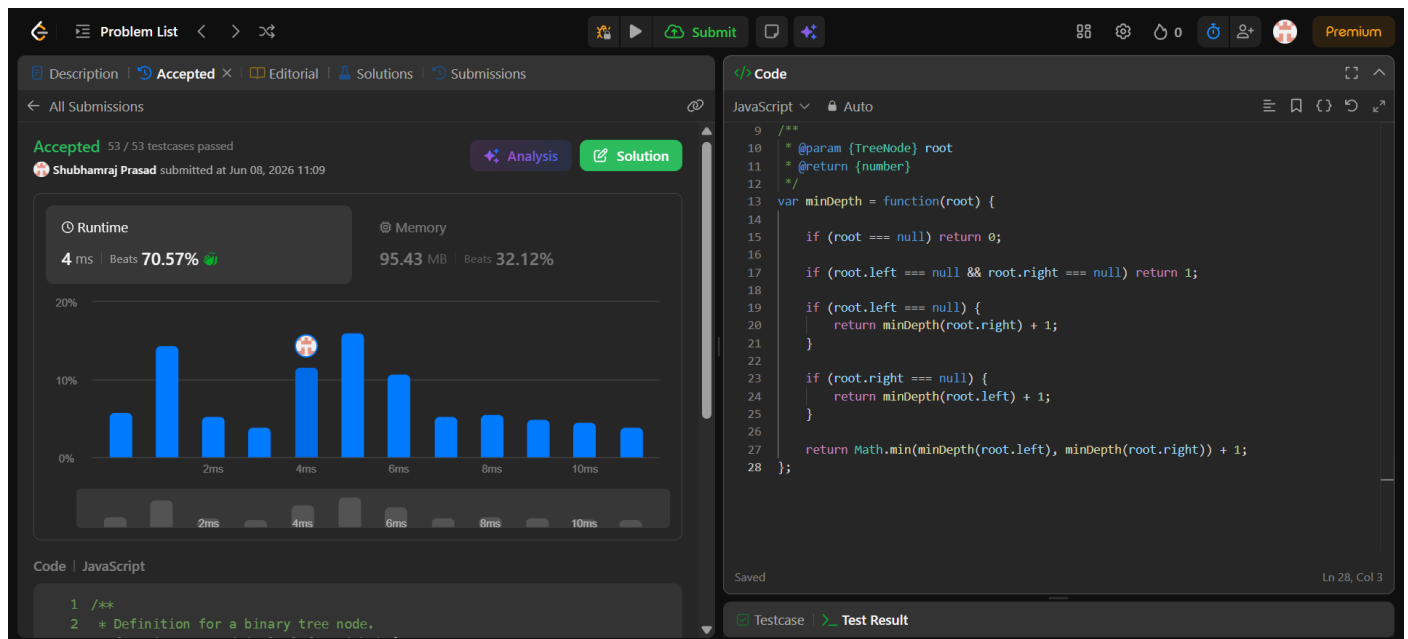
Description: -

- **Time Complexity:** $O(n)$ — Every element is pushed and popped from the stack at most once.
- **Space Complexity:** $O(n)$ — To store the elements in the stack and the map lookup

Question 5:- <https://leetcode.com/problems/minimum-depth-of-binary-tree/description/>

Solution: - <https://leetcode.com/problems/minimum-depth-of-binary-tree/submissions/2025974021/>

Screenshot:-



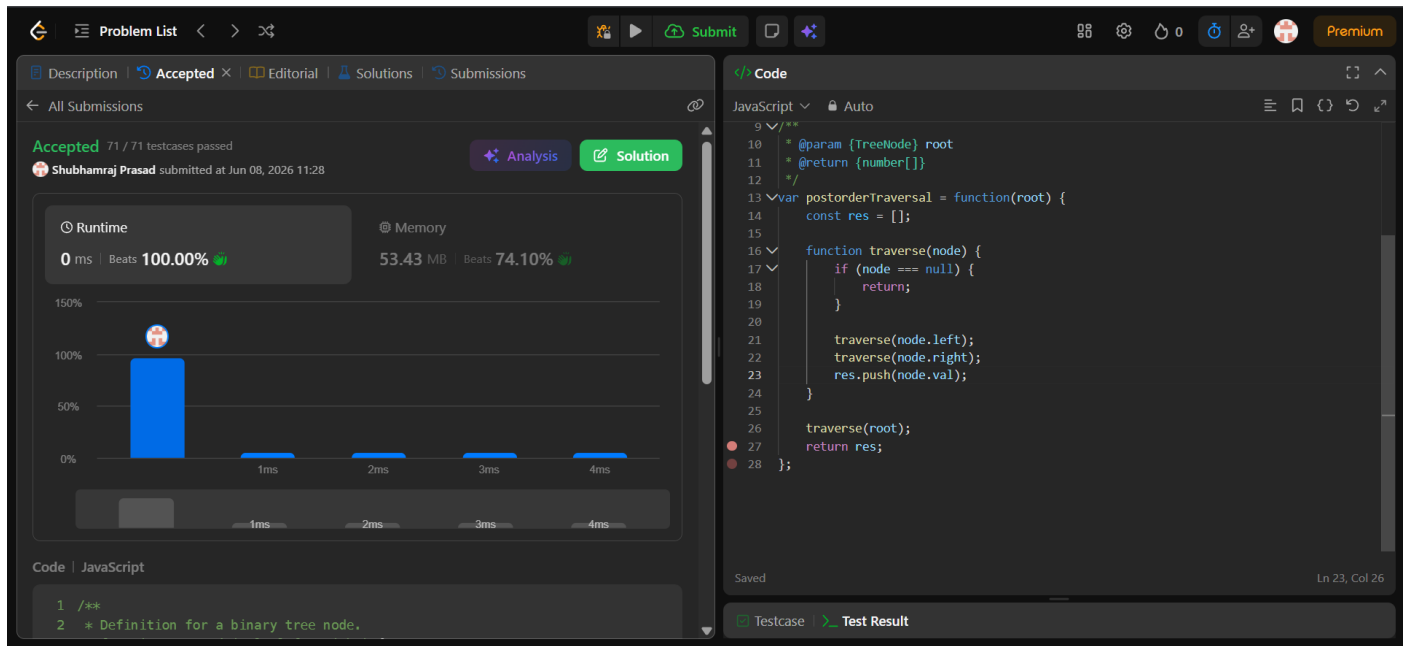
Description: -

- **Time Complexity:** $O(n)$ - You must visit every node exactly once in the worst case.
- **Space Complexity:** $O(h)$ - Uses memory on the call stack proportional to the tree's height ($O(n)$ for a straight-line tree, $O(\log n)$ if perfectly balanced).

Question 6:- <https://leetcode.com/problems/binary-tree-postorder-traversal/>

Solution: - <https://leetcode.com/problems/binary-tree-postorder-traversal/submissions/2025993893/>

Screenshot:-



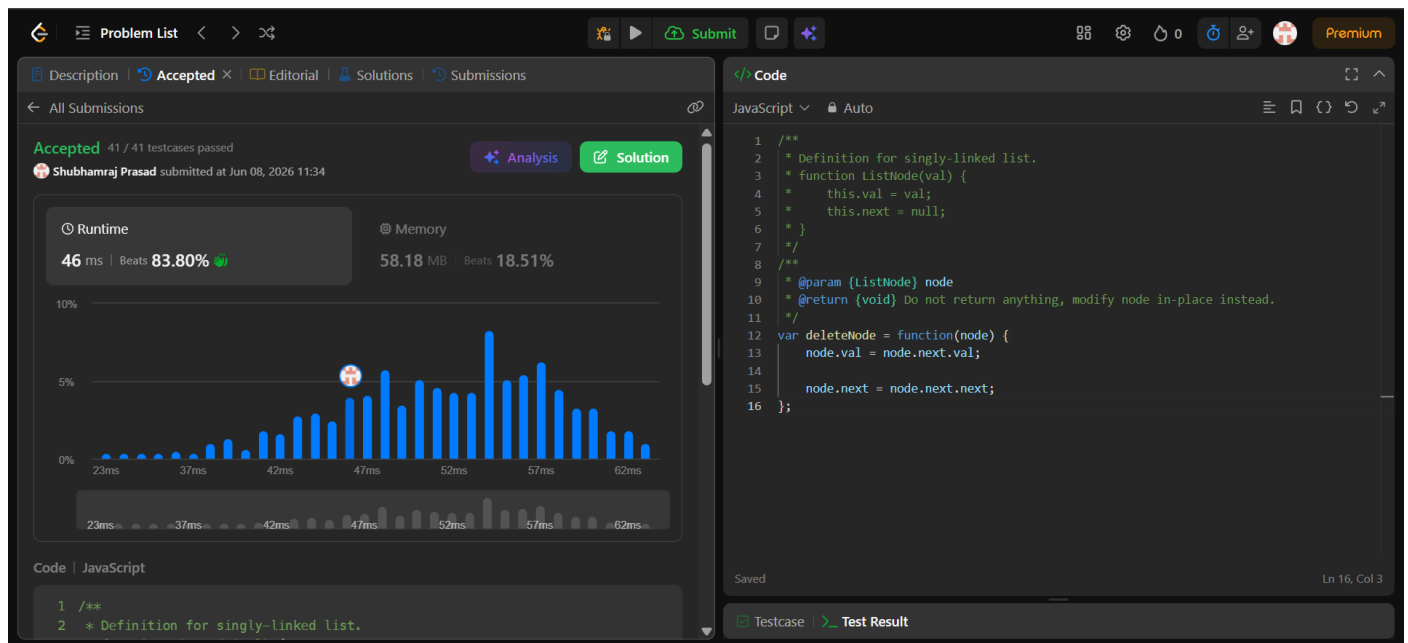
Description: -

- **Time Complexity:** $O(n)$ => The algorithm visits every single node in the binary tree exactly once. Inside each node, it only performs quick, constant-time operations (checking for null and pushing a value to an array).
- **Space Complexity:** $O(n)$ =>
 - 1. **Output:** The result array stores all n node values, using $O(n)$ space.
 - 2. **Call Stack:** In the worst-case scenario (a skewed tree that looks like a single long line), the recursive call stack will grow to a depth of n functions. In a perfectly balanced tree $O(\log n)$, but the total space remains $O(n)$ due to the output array.

Question 7:- <https://leetcode.com/problems/delete-node-in-a-linked-list/description/>

Solution: - <https://leetcode.com/problems/delete-node-in-a-linked-list/submissions/2026000037/>

Screenshot:-



Description: -

- **Time Complexity:** $O(1)$ => It takes a constant amount of time since we only perform two pointer/value reassignments.
- **Space Complexity:** $O(1)$ => No additional memory is allocated.